

Introducción y Alcance del Curso

Sobre el Alcance

Este curso será abordado de forma teórica-práctica, teniendo en cuenta que alrededor de un 50-80% de las lecciones serán magistrales con laboratorios de práctica donde se deberán realizar los ejercicios. Durante las lecciones sincrónicas, se verán los conceptos y se explicarán las minutas técnicas y de investigación asociadas a cada tema. Por otra parte, en el tiempo asincrónico, se deberán ejecutar las prácticas que cubrirán los conceptos.

Los conceptos principales a ver son:

- Concepto de ESL
- Modelos de cómputo y de lenguaje
- Modelos de arquitectura de alto nivel
- Verificación de modelos de arquitectura de alto nivel
- Diseño y descripción usando HLS

Sobre la Evaluación

Este curso será evaluado de la siguiente manera:

- **4 proyectos cortos** al final de cada tema, que consisten en poner en práctica los distintos conceptos con problemas didácticos.
- **1 proyecto final** que consiste en la confección de un sistema computacional que acelere algún cómputo o implemente un sistema completo. El tema es seleccionado por cada grupo. Los proyectos se ejecutarán en grupos de 4 personas.
- **1 prueba conceptual** oral para verificación de contenidos.

Una aclaración importante del proyecto final es que debe ser novedoso respecto del estado del arte y desembocar en una contribución publicable mediante un artículo científico en una conferencia. Lo ideal es que tenga relación con su proyecto final de tesis, si es posible.

Sobre el Contacto con el Profesor

Las comunicaciones serán oficialmente por el TEC Digital. Para efectos no oficiales, se pone a disposición un servidor en Discord.

Modelos de Computación y de Lenguajes

Como punto de partida, tenemos que partir de la introducción del curso (Diseño Electrónico): el proceso de diseño consiste principalmente:

- Conjunto de requerimientos a ser satisfechos

Los requerimientos permiten que el diseño nazca. Definen de forma abstracta la funcionalidad de un dispositivo electrónico, sus entradas, sus salidas y su comportamiento.

- Definición de una interfaz exterior

Los dispositivos electrónicos definen entradas y salidas para interactuar con otros circuitos electrónicos o con el mundo exterior.

- Definición de un comportamiento

Se define cómo reacciona el dispositivo ante las entradas y cómo debe producir las salidas.

- Definición de una implementación

Se define cómo se organizan los sistemas internos de un dispositivo: cómo se dividen los módulos, cómo se interconectan y su grado de granularidad.

Electronic System Level Design (ESL)

A partir de estas definiciones generales, podemos definir el Diseño Electrónico de Alto Nivel como una metodología de diseño emergente que se enfoca en un alto grado de abstracción y cuyo entregable principal son los modelos.

Esto sigue la misma idea de los modelos físicos. Por ejemplo, podemos definir el modelo general de desplazamiento de un objeto mediante las leyes fundamentales de Newton:

$$\mathbf{F} = m \frac{d\mathbf{v}}{dt} = m\mathbf{a}$$

Sin embargo, podemos llegar a complicar más el modelo, introduciendo las distintas fuentes de aceleración, la distribución de masa o, inclusive, la introducción de la física cuántica:

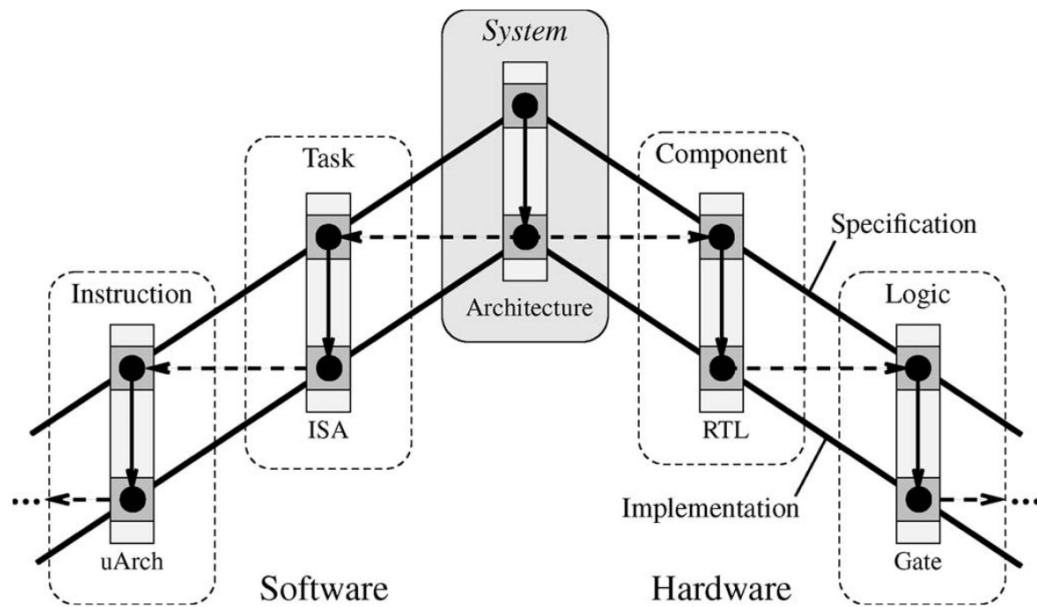
$$i\hbar \frac{\partial}{\partial t} \Psi(x, t) = \left[-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x, t) \right] \Psi(x, t).$$

Entonces, partiendo de este ejemplo en particular, podemos extrapolarlo a la parte electrónica, donde no describimos un circuito digital a nivel de la física de los transistores, sino mediante una abstracción instrumental basada en el álgebra booleana.

Para ESL, buscamos simplificar aún más la descripción de una arquitectura, concentrándonos en cómo interactúa el dispositivo con otros (a nivel transaccional), lo que permite construir modelos más rápido.

Un modelo con alto nivel de abstracción, en el contexto del ESL, permite tener:

- Un modelo de comportamiento de caja negra que permite desarrollar la periferia
- Un modelo de comportamiento de caja negra que permite desarrollar el hardware
- Un modelo de comportamiento de caja negra que permite desarrollar el firmware



Flujo de ESL. En este caso, la especificación del modelo de sistema (centro) permite el desarrollo en ambas direcciones. Obtenido de: Gerstlauer et al. 2009. Electronic System-Level Synthesis Methodologies.

En este curso, estaremos ahondando en las distintas herramientas de ESL, así como en el proceso de co-diseño de hardware y software, donde el ESL permite el desarrollo paralelo de las mencionadas partes.

Modelos de Computación

Los modelos de abstracción de alto nivel de ESL eliminan ciertas especificaciones de implementación en torno al tiempo. Los modelos de computación son, particularmente:

- Sincrónicos o reactivos: las actualizaciones de todos los modelos se realizan de acuerdo con un reloj global. Esto es utilizado en software crítico.
- Eventos: las actualizaciones de los módulos se realizan en intervalos discretos.
- Flujo de datos: los módulos se intercomunican y reaccionan de acuerdo a un intercambio de mensajes con datos: pueden ser sincrónicos o basados en FIFO.

Con respecto a la precisión de tiempo, existen tres formas de abstraerlo:

1. Funcional/Untimed: no utiliza la noción de tiempo y sólo se describe el comportamiento funcional.
2. Loosely-Timed: se basa en las transacciones de mensajes y no se enfoca en señales individuales.
3. Ciclo exacto: utiliza una señal que gobierna la activación de las señales en el tiempo.

Modelos de Lenguajes

En software, tenemos distintas variables sobre tipos de lenguajes, como procedurales, declarativos, funcionales y meta lenguajes. En el contexto de la descripción de modelos, nos podemos basar en describir un modelo utilizando herramientas de alto nivel existentes.

Lenguajes Imperativos

Describen cómo ejecutar una tarea paso a paso. El modelo se construye mediante:

- Secuencias de instrucciones-
- Cambios de estado.
- Variables mutables.
- Flujo de control explícito.

Son muy usados cuando se modela comportamiento temporal, simulación, algoritmos de control y arquitecturas HW/SW.

Un ejemplo es SystemC, que es un framework que se basa en el modelo de computación transaccional.

```
void cpu::execute() {  
    while(true) {  
        instr = bus.read(pc);  
        decode(instr);  
        pc++;  
        wait(10, SC_NS);  
    }  
}
```

En este caso, el flujo es explícito, con un lazo. También existe estado (pc) y el tiempo se modela con wait().

En ESL, se utilizan principalmente para modelos de referencia (HLS/SystemC) o bien para simulación rápida (como en Verilator). Sin embargo, el problema es que su nivel de abstracción es menor contra otras alternativas y tienden a tener mayores errores de estado.

Lenguajes Declarativos

Describen qué debe cumplir el sistema, no cómo implementarlo. El énfasis está en:

- Restricciones
- Relaciones
- Propiedades
- Reglas

Son muy usados en verificación, síntesis, especificaciones formales y resolución de restricciones.

Un ejemplo particular es el uso de XML y UML para relacionar módulos, Makefiles. Otro ejemplo es SMT, usado en verificación formal:

```
(assert (= out (+ a b)))
```

Este es otro con UML:

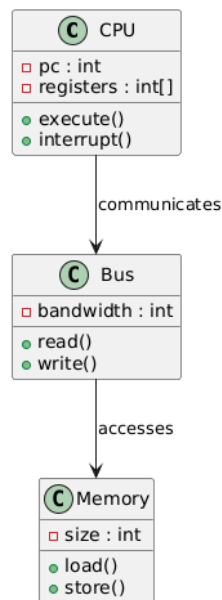
```
@startuml
class CPU {
- pc : int
- registers : int[]
+ execute()
+ interrupt()
}

class Bus {
- bandwidth : int
+ read()
+ write()
}

class Memory {
- size : int
+ load()
+ store()
}

CPU --> Bus : communicates
Bus --> Memory : accesses
@enduml
```

El código anterior en PlantUML genera el siguiente diagrama de relaciones:



Lenguajes Funcionales

Describen el sistema como evaluación de funciones matemáticas. Se enfocan en:

- Inmutabilidad: las funciones y los atributos internos no cambian
- Ausencia de efectos secundarios: ya que los objetos no cambian internamente
- Composición funcional: permiten componer funciones complejas mediante encadenamiento.

Se utilizan ampliamente en especificación formal, transformaciones, generación de hardware, modelos concurrentes; ya que cuentan con las siguientes características:

- Datos inmutables
- Funciones puras
- Alta abstracción matemática
- Facilitan razonamiento formal

El detalle de la formalidad es que lo hacen útil para razonamiento de verificación formal usando frameworks matemáticos.

```
# Función para aplicar una operación a todos los elementos de una lista
def apply_operation(nums, operation):
    return list(map(operation, nums))

# Uso de la función con una lambda para elevar al cuadrado cada elemento de la lista
nums = [1, 2, 3, 4, 5]
squared_nums = apply_operation(nums, lambda x: x**2)
print(squared_nums) # imprime [1, 4, 9, 16, 25]
```

El estilo funcional a menudo se representa mediante funciones anónimas, que permiten componer funciones.

Meta lenguajes

Son lenguajes usados para definir otros lenguajes, describir sintaxis y construir frameworks de modelado. En ESL son fundamentales porque muchos entornos usan:

- DSLs (Domain Specific Languages).
- Modelos automáticos.
- Generación de código.

En otros casos, los meta lenguajes son parte de lenguajes existentes, como en C++:

```
template <typename T, int NT, int N>
struct AddPairs {
    static T Execute(const T vec[NT], int idx = 0) {
#pragma HLS INLINE
        T lhs = AddPairs<T, NT, N / 2>::Execute(vec, idx);
        T rhs = AddPairs<T, NT, N / 2>::Execute(vec, N / 2 + idx);
        return lhs + rhs;
    }
};

template <typename T, int NT>
struct AddPairs<T, NT, 1> {
    static T Execute(const T vec[NT], int idx) {
        return vec[idx];
    }
};
```

Por otra parte, también componen sistemas enteros, usando XML

```
<?xml version="1.0" encoding="UTF-8"?>

<soc name="Simple_RISCv_SoC">

  <cpu name="rv32_core" isa="RV32I" frequency="100MHz"/>

  <bus name="axi_bus" type="AXI4"/>

  <memory name="main_ram" size="64KB" type="SRAM"/>

  <peripheral name="uart0" type="UART"/>

  <connections>
    <link from="rv32_core" to="axi_bus"/>
    <link from="axi_bus" to="main_ram"/>
    <link from="axi_bus" to="uart0"/>
  </connections>

</soc>
```

Un ejemplo es el uso de LiTeX o de [FuseSoC](#).

Tarea 1:

Para la próxima semana:

1. Preparar el entorno en la computadora personal para usar SystemC.
2. Preparar el entorno en la computadora personal para usar PipelineC.
3. Preparar el entorno en la computadora personal para usar Spade.
4. Definir grupos de trabajo de 4 personas y comunicarlos mediante el Discord (antes del domingo 17 de mayo).